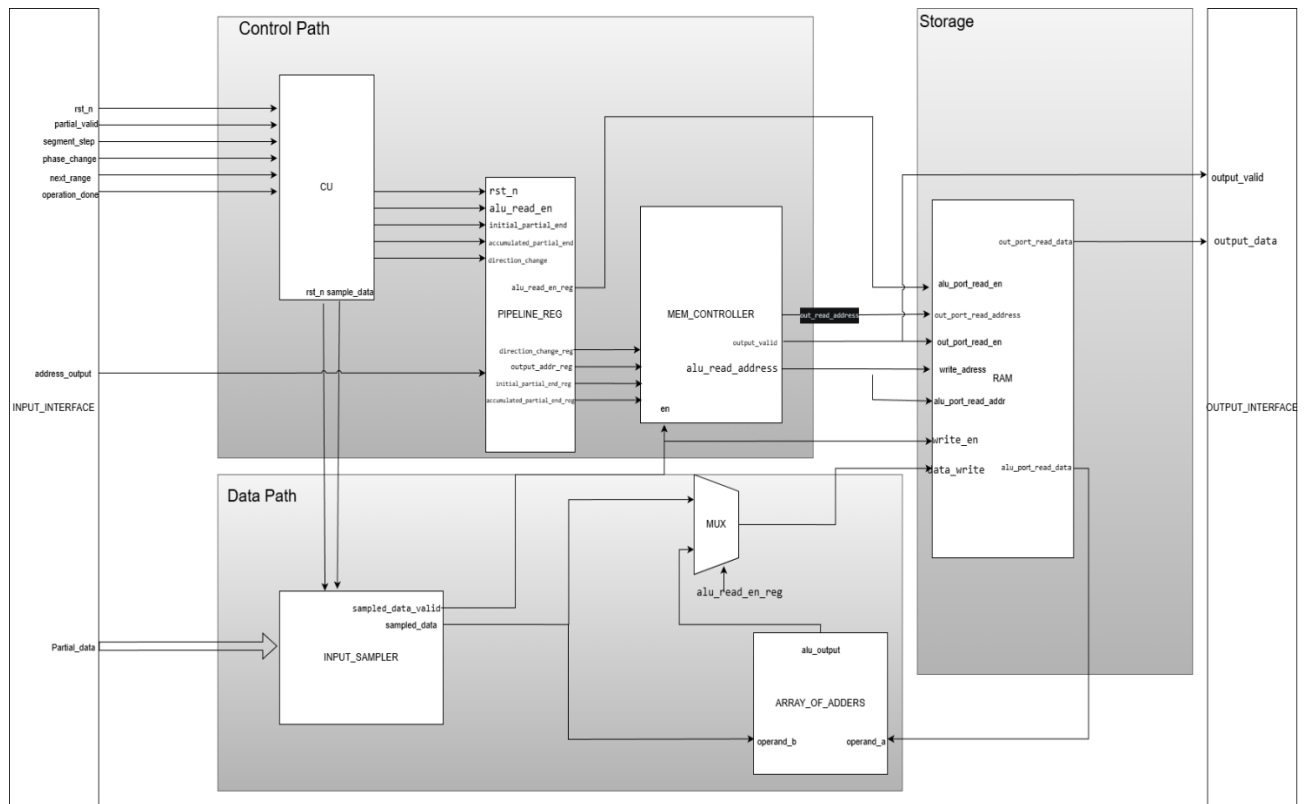


Design Brief

Contents

Architecture.....	2
Components	2
Input sampler	2
Array Of adders.....	2
Memory Controller	3
Control Unit	3
RAM	3
Performance:	4
Throughput:.....	5
Latency:.....	5
Utilization	5

Architecture



Components

Input sampler

Its main rule is to sample the input data based on a control signal called sample_data from the control unit, then it receives data and concatenate the incoming data into local register of size equal to the output word.

Array Of adders

This module using generate statement to generate a number of parallel adders equal to the number of **OUTPUT_DATA_ELEMENTS** given parameter.

Its main function is to adding two output words element wise

Memory Controller

The functions of this module are:

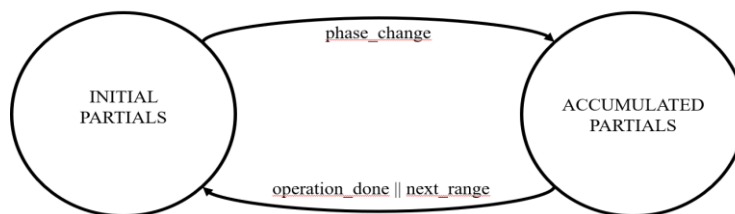
- Manage the address of the current range being processed whether its initial or accumulating with handling to the change phase operation.
- Translating logical address of the downstream size to physical address of the actual memory.
- Tracing valid ranges using valid array mask and mark the oldest range in order to stream correct data to the downstream side when needed.
- Tracing time out for each valid range using counter per range.
- Handle the deassertion of valid ranges if readied by the downstream or got a timeout and make it available for other ranges to be received.

The unit is fully parameterized it create the suitable size of counters, valid mask depth, number of ranges and their sizes and Widths of the registers using the provided parameters in the specs file.

Control Unit

Its main rule is to use the control signals provided by the upstream side to manage generating required signals for other modules.

It uses 2 state FSM initial & Accumulate to correctly perform the required mode function.



RAM

We choose that the memory will have one synchronous write port an two asynchronous read ports one for the output and one for internal accumulation of segments, this method makes the design able to simultaneously provide data for the downstream while processing and received other ranges from the upstream side.

The memory size is parameterized based on the designer option but we choose 1024 depth and 128 wide which is 16 KB this size enables us to have 5 ranges in our memory with minimum waste of memory locations.

Challenges

- 1- case of tracking the oldest range sent, this problem was solved by using a pointer that points to the oldest valid range in the valid mask.

The mechanism is simple the pointer initially pointing at the location of the first range where the first valid range will be surely inserted, then after this if the location where the pointer is pointing at dropped to 0 then the pointer should look for the next oldest range so the pointer increment is self until it find the first valid in the mask.

- 2- Memory wasting logic, we noticed that with the current parameter of $\text{MAX_SEGMENT_PER_RANGE} = 100$ & $\text{OUT_PER_SEG} = 2$ then the range have a size of 200 location, so to effectively choose the RAM size we calculated memory waste size.

In case of 512 we can only have 2 ranges with 112 wasted locations.

In case of 1024 we can have 5 ranges with only 24 wasted location , by doubling memory we gained extra range with 21% decrease in memory waste.

- 3- Parametrizing the design we have to make every single line parametrized so we used generate statements in the adders and counters to generate the suitable number of parallel adders and counter in order to perform as expected.

Test cases:

- Test single range & read output
- Test single range with 1 time accumulation & read output
- Test single range with 3 time accumulation & read output
- Test multiple ranges and read output.

Performance:

Our design is fully parametrized it can work with various number of ranges with custom memory size, and different output and partial sizes.

The design using generic generate statements across the design in order to achieve interpretability to various system with different requirements and different data sizes.

Using the default values of the parameters given in the specs file we synthesis out design on fpga ZYNQ-7 ZC706 Evaluation Board (xc7z045ffg900-2) with 200MHz frequency.

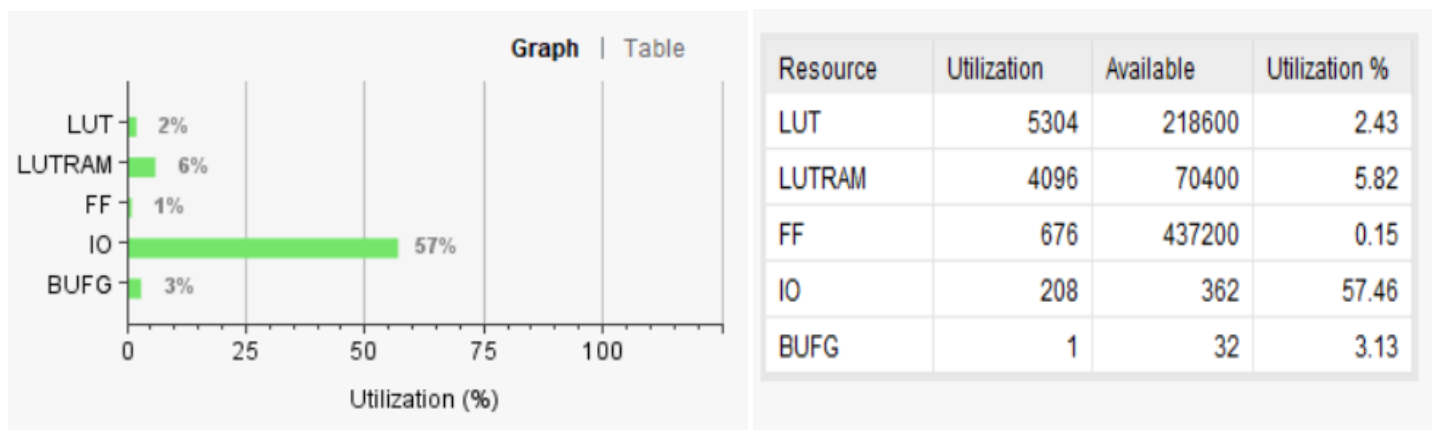
Throughput:

Because we have asynchronous read our Throughput is 1 Output Word / cycle.

Latency:

From the moment the final partial is received the link takes **3 cycles** to make the range ready for reading by the downstream side

Utilization



Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.056 ns	Worst Hold Slack (WHS): 0.069 ns	Worst Pulse Width Slack (WPWS): 1.732 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 37574	Total Number of Endpoints: 37574	Total Number of Endpoints: 4794
All user specified timing constraints are met.		

No timing violation under 200 MHZ implementation.